

desktop-adapter

Desktop Adapter

The desktop adapter provides an interface for testing desktop applications, particularly those built with web-based frameworks (Tauri, Wails, Electron) that expose a WebView for UI interaction and an IPC mechanism for backend communication.

DesktopAdapter Interface

Defined in `adapter_desktop.go`:

```
type DesktopAdapter interface {
    LaunchApp(ctx context.Context, config DesktopAppConfig) error
    IsAppRunning(ctx context.Context) (bool, error)
    Navigate(ctx context.Context, url string) error
    Click(ctx context.Context, selector string) error
    Fill(ctx context.Context, selector, value string) error
    IsVisible(ctx context.Context, selector string) (bool, error)
    WaitForSelector(ctx context.Context, selector string, timeout
        time.Duration) error
    Screenshot(ctx context.Context) ([]byte, error)
    InvokeCommand(ctx context.Context, command string,
        args ...string) (string, error)
    WaitForWindow(ctx context.Context, timeout time.Duration)
        error
    Close(ctx context.Context) error
    Available(ctx context.Context) bool
}
```

Method Summary

Method	Purpose
LaunchApp	Start the desktop application with the given configuration
IsAppRunning	Check if the application process is alive
Navigate	Load a URL in the application's WebView
Click	

Method	Purpose
	Click an element by CSS selector in the WebView
Fill	Type a value into an input by CSS selector
IsVisible	Check if an element is visible in the WebView
WaitForSelector	Block until a WebView element appears or timeout expires
Screenshot	Capture the application window as a PNG byte slice
InvokeCommand	Send an IPC command to the app's backend and return the response
WaitForWindow	Block until the application window is ready
Close	Shut down the application and release resources
Available	Check if the testing environment is ready

Configuration Type

DesktopAppConfig

```

type DesktopAppConfig struct {
    BinaryPath string      `json:"binary_path"`
    Args       []string    `json:"args"`
    WorkDir    string      `json:"work_dir"`
    Env        map[string]string `json:"env"`
}

```

TauriCLIAdapter

The built-in implementation uses the W3C WebDriver protocol to control a Tauri application. The adapter manages the full lifecycle: launching the binary, creating a WebDriver session, interacting with the WebView, and shutting down.

Architecture

Go Challenge --> TauriCLIAdapter --> WebDriver HTTP --> Tauri App

|

WebView (HTML)

|

Backend (IPC)

Constructor

```
adapter := userflow.NewTauriCLIAdapter("/path/to/app-binary")
```

Launch Sequence

When `LaunchApp` is called:

1. The adapter finds a free TCP port on the local machine.
2. The binary is started with environment variables:
 - `TAURI_AUTOMATION=true` -- enables WebDriver support in Tauri.
 - `TAURI_WEBDRIVER_PORT=<port>` -- tells Tauri which port to listen on.
 - Any additional env vars from `DesktopAppConfig.Env`.
3. A goroutine monitors the process exit via `cmd.Wait()`.

WebDriver Session

After launch, `WaitForWindow` must be called to establish the WebDriver session:

1. Polls the WebDriver endpoint every 500ms.
2. Sends `POST /session` with empty capabilities.
3. Parses the session ID from the response.
4. All subsequent WebView interactions use this session ID.

WebView Interaction

All UI methods (`Click`, `Fill`, `IsVisible`, `WaitForSelector`) work through the WebDriver protocol:

- **Find element:** `POST /session/{id}/element` with CSS selector.
- **Click:** `POST /session/{id}/element/{elem}/click`.
- **Fill:** `POST /session/{id}/element/{elem}/clear` then `POST /session/{id}/element/{elem}/value`.
- **Is displayed:** `GET /session/{id}/element/{elem}/displayed`.
- **Navigate:** `POST /session/{id}/url`.
- **Screenshot:** `GET /session/{id}/screenshot` (returns base64 PNG).

IPC Commands

`InvokeCommand` executes JavaScript via the WebDriver async script endpoint to call `window.__TAURI__.invoke()`:

```
result, err := adapter.InvokeCommand(ctx, "get_config",
    `{"key":"theme"}`)
```

This generates:

```
return JSON.stringify(await window.__TAURI__.invoke('get_config',
    {"key":"theme"}))
```

Shutdown

Close() performs:

1. Deletes the WebDriver session (DELETE /session/{id}).
2. Kills the process.
3. Waits for the process goroutine to complete.

Availability

Available() checks if the binary path exists on disk via os.Stat.

Example: Desktop Launch Challenge

```
adapter := userflow.NewTauriCLIAdapter("/path/to/my-app")
```

```
challenge := userflow.NewDesktopLaunchChallenge(
    "CH-DESKTOP-001",
    "App Launch",
    "Launch the desktop app and verify it remains stable",
    nil,
    adapter,
    userflow.DesktopAppConfig{
        BinaryPath: "/path/to/my-app",
    },
    10*time.Second,
)
```

The challenge:

1. Launches the app.
2. Waits up to 10 seconds for the window.
3. Waits the stability period.
4. Checks if the app is still running.
5. Takes a screenshot.
6. Closes the app.

Example: Desktop WebView Flow

DesktopFlowChallenge reuses the BrowserFlow type to drive interactions in the app's WebView:

```
flow := userflow.BrowserFlow{
    Name:      "settings-flow",
    StartURL:  "tauri://localhost/settings",
    Steps: []userflow.BrowserStep{
        {Name: "click-theme", Action: "click", Selector: "#theme-toggle"},
        {Name: "verify-dark", Action: "assert_visible", Selector: ".dark-mode"},
        {Name: "capture", Action: "screenshot"},
    },
}

challenge := userflow.NewDesktopFlowChallenge(
    "CH-DESKTOP-002",
    "Settings Flow",
    "Toggle dark mode and verify",
    []challenge.ID{"CH-DESKTOP-001"},
    adapter,
    flow,
)
```

Supported actions for desktop flows: navigate, click, fill, wait, assert_visible, screenshot.

Example: IPC Commands

```
commands := []userflow.IPCCommand{
    {
        Name:      "get-version",
        Command:    "get_version",
        ExpectedResult: "1.0",
    },
    {
        Name:      "set-config",
        Command:    "set_config",
        Args:      []string{"{"theme":"dark"}"},
        Assertions: []userflow.StepAssertion{
            {Type: "not_empty", Target: "response", Message: "config response should not be empty"},
        },
    },
}

challenge := userflow.NewDesktopIPCChallenge(
    "CH-DESKTOP-003",
    "IPC Commands",
)
```

```
"Verify backend IPC commands respond correctly",  
[]challenge.ID{"CH-DESKTOP-001"},  
adapter,  
commands,  
)
```